# SpareDukaan Website Project Documentation

## Business concept and implementation requirements

Version 1.0 | 25 October 2026 | Prepared for the SpareDukaan founder and delivery team

SpareDukaan is a proposed online store for mobile spare parts, accessories, and repair tools. Its central promise is to help customers identify the correct part for their exact device, understand the available options, and receive a clearly described product with dependable order support.

This document defines what to build, why it matters, the information the business must supply, how the system should work, and how to determine whether it is ready for launch. It is intended for the founder, designer, developer, catalogue manager, and fulfilment team.

The recommended first release is a branded, mobile-first retail store using the Device First design direction. Shopify is the proposed commerce platform, with a custom device finder and structured compatibility data. Platform selection, operating budget, supplier arrangements, and the balance between retail and wholesale remain business decisions.

### Confirmed direction and present status

| Item | Status |
|---|---|
| Business name | SpareDukaan selected by the founder |
| Domain | sparedukaan.com proposed; purchase and ownership not confirmed |
| Visual direction | Device First selected in the design comparison |
| Existing website | Private Maxbhi-style demonstration with browsing, search, and a browser-local cart |
| Production store | Not yet built; live prices, stock, accounts, payments, shipping, and returns are not connected |
| Intended launch market | India is the working assumption; founder confirmation required |

The demonstration is a design reference, not an operational store or a source of verified supplier inventory. The SpareDukaan production site must use its own business identity, policies, authorised images, and verified product data.

# Contents

1. Idea and business goals / page 3

2. Customers and release scope / page 4

3. Design and website structure / page 5

4. Customer journeys / page 6

5. Storefront requirements / page 7

6. Order and administration requirements / page 8

7. Architecture and integrations / page 9

8. Core data relationships / page 10

9. Catalogue data dictionary / page 11

10. Order and inventory data / page 13

11. Data sourcing and publication / page 14

12. Operations and responsibilities / page 15

13. Security and quality requirements / page 16

14. Testing and launch acceptance / page 17

15. Delivery plan and business decisions / page 18

16. Measurement and operating reviews / page 19

17. References and glossary / page 20

# 1 Idea and business goals

## The problem to solve

Customers often know that a screen, battery, or charging port needs replacement but cannot confidently identify the correct part. Similar phone names, regional model variants, connector differences, panel technologies, and incomplete supplier descriptions make selection difficult. A visually similar component is not sufficient evidence that it will fit.

SpareDukaan should reduce this uncertainty before checkout. Its product finder should narrow the catalogue by exact device, its product pages should explain meaningful differences, and its support process should help resolve cases that cannot be confirmed automatically.

## The proposed business

The initial business model is a single merchant selling sourced products directly to customers. Revenue comes from product sales and, where applicable, disclosed shipping charges. A third-party seller marketplace, repair booking service, and device resale business are outside the first release.

Start with a manageable, verified selection of phone models and parts. The founder should choose the opening assortment using supplier reliability, demand evidence, stock availability, expected margin, and the ability to support returns. A broad catalogue with uncertain compatibility should not be a launch objective.

## Goals

- Help a customer find a compatible SKU using a brand, model name, exact model code, or part number.
- Explain specifications and product condition before purchase so expectations match the delivered item.
- Keep sellable stock, price, payment status, and order status accurate.
- Give the team a repeatable way to receive stock, fulfil orders, and resolve replacement or refund requests.
- Establish a recognisable SpareDukaan brand that can later serve repair professionals as well as retail buyers.

## Commercial controls

Track contribution per fulfilled order: net product revenue less product cost, payment fees, shipping subsidy, packaging, discounts, and expected return or replacement costs. Taxes collected should not be treated as margin. Exact accounting and tax treatment must be configured with the business's adviser.

Do not promise free shipping, fixed delivery times, original-manufacturer parts, or a standard warranty until the business can substantiate those claims. Each promise should be backed by a configured service rule or product-level record.

## What success means

A successful launch allows a real customer to discover a correct part, understand its specification and delivery conditions, pay through the selected provider, receive tracking, and obtain support. The team must be able to reconcile the payment, inventory movement, shipment, and any refund for that order.

# 2 Customers and release scope

## Customer groups

| Group | Primary need | Product response |
|---|---|---|
| Individual buyer | Identify the correct part and understand what is included | Guided device selection, clear photos, compatibility explanation, simple checkout |
| Repair technician | Find a specific SKU quickly and reorder reliably | Model-code search, precise specifications, saved orders, later quick ordering |
| Repair shop buyer | Purchase multiple parts with predictable availability | Later bulk ordering, approved trade pricing, invoices, stock visibility |
| Store operator | Maintain accurate catalogue and fulfil orders | Controlled imports, review queues, inventory records, order administration |

Retail-first is a planning assumption. If repair shops are the main customers, move quick ordering and approved trade pricing earlier in the roadmap and validate the selected platform and plan against those needs.

## First release

The first production release should include the SpareDukaan identity, responsive homepage, exact-device finder, search, category and model pages, product details, cart, checkout, payment integration, shipping integration, basic customer order access, support, and a documented returns process. Staff must be able to edit catalogue data, manage stock, process orders, and review operational exceptions.

## Later releases

Add wholesale applications and pricing rules, bulk order entry, saved workshop lists, automatic restock notifications, multilingual content, richer repair guidance, and multiple stock locations only when business demand and operating capacity justify them. Provider and plan support for these features must be validated before commitment.

## Excluded from the initial release

• Multi-seller marketplace onboarding, commissions, and settlement.

• Native Android and iOS applications.

• Repair-service booking and technician dispatch.

• Automatic claims that an unverified component is compatible.

• Custom storage or processing of raw payment card details.

• Copying another merchant's entire catalogue, customer reviews, policies, or commercial claims.

## Current prototype compared with production scope

The private demonstration supports a limited reference catalogue, category and brand browsing, basic search, product previews, and a cart stored in the current browser. It does not authenticate customers, create live orders, calculate real prices, reserve inventory, collect payment, send newsletters, or fulfil shipments. The Device First concept is a separate design exploration and has not replaced that hosted demonstration.

# 3 Design and website structure

## Visual direction

Use the Device First concept: a warm white background, deep green actions, dark readable text, restrained borders, and product photography with consistent framing. The homepage should devote its most prominent space to selecting or searching for a device. Use the SpareDukaan wordmark and business-owned imagery throughout the production site.

Use green for actions and confirmed states, with text or icons to explain meaning. Error and warning states must remain understandable without colour. Keep decorative graphics secondary to model selection, product specifications, and purchase information.

## Main navigation

Home; Find parts; Phone brands; Screens; Batteries; Charging parts; Cameras and other parts; Repair tools; Order tracking; Help; Account; Cart. Accessories may be a separate category if supported by the opening assortment. Avoid adding unrelated categories merely because they appeared on the reference site.

## Page hierarchy

| Page | Main content | Primary action |
|---|---|---|
| Home | Device finder, useful categories, product discovery, support entry | Select a device |
| Brand and model directory | Searchable device names and exact model codes | Choose the correct model |
| Model page | Parts compatible with the selected device | Choose a part category |
| Category or search results | Relevant products, specifications, filters, availability | Compare or open a product |
| Product details | Images, selected variant, compatibility, price, stock, delivery, warranty | Add the verified variant to cart |
| Cart and checkout | Line items, quantities, selected device, charges, delivery and payment | Place an order |
| Account and tracking | Orders, shipment progress, invoices, support access | Track or request support |
| Help and policies | Contact details, model identification, shipping, returns and privacy | Resolve a question |

## Product page requirements

Show the exact SKU and variant being purchased. Separate display technology, frame inclusion, colour, condition, manufacturer attribution, and included tools or adhesives. Use supplier-supported specification terms. Labels such as premium or original must have a defined, defensible meaning.

Display compatibility against the selected model code. If compatibility is unknown, do not show a confirmed-fit badge. Show stock status, delivery estimate assumptions, total price information, warranty conditions, and return restrictions before the final purchase action.

### Mobile behaviour

Keep search and selected-device context easy to reach. Use readable input sizes, large touch targets, an accessible filter panel, and a sticky purchase action that does not cover essential content. Preserve the user's device selection when moving between categories. Allow a separate selected device for each cart line where necessary.

# 4 Customer journeys

### A buyer knows the phone model

1. The buyer selects a brand and model or enters a model code.
2. The site resolves the device to a specific model-code record. Ambiguous names show a disambiguation step.
3. The buyer selects a part category and sees compatible products.
4. The buyer compares specifications and chooses a variant.
5. The product page repeats the compatible device, included components, price, stock status, and delivery conditions.
6. The cart preserves the chosen SKU and device context. Checkout validates current price and availability before payment.

### A buyer is unsure of the model

Provide instructions for finding the model code in device settings or on appropriate packaging. Search results may offer likely device names, but suggestions must not be treated as verified compatibility. If uncertainty remains, provide a support path and explain what information is needed. Request only relevant details; never require an IMEI merely to browse the catalogue.

### A repair professional knows the part

Allow direct search by SKU, manufacturer part number, or exact model code. Display technical details without forcing the buyer through the homepage finder. In a later trade experience, allow quantities for multiple items and reordering from previous purchases while rechecking price, compatibility, and stock.

### Payment does not finish cleanly

If the payment screen closes or the result is delayed, show a pending state and provide a safe order-status route. Do not automatically ask for a second payment while the first payment is unresolved. The platform or integration must verify the provider outcome and reconcile late notifications. A browser success message alone must never authorise fulfilment.

### An item cannot be delivered or must be returned

An unserviceable postcode should be detected before payment wherever the chosen integrations support it. If a carrier later rejects the shipment, the order enters a staff exception queue. Return requests should capture the order item, reason, relevant evidence, and requested resolution. Staff review eligibility, record the decision, and track the replacement or refund separately.

### Essential exception states

Design explicit states for no search matches, unsupported models, uncertain compatibility, out-of-stock variants, price changes, insufficient stock, invalid address, unavailable delivery, payment failure, pending payment, delayed shipment, and failed refund. Each state should offer an accurate next action and preserve the customer's work where possible.

# 5 Storefront requirements

The following identifiers can be used in development tickets and acceptance tests. All listed requirements belong to the first release unless a row explicitly states otherwise.

| ID | Requirement | Acceptance condition |
|---|---|---|
| FR01 | Brand and device finder | The selected brand limits available models; model aliases resolve to a canonical device record |
| FR02 | Exact compatibility | A confirmed-fit result is shown only for a verified SKU and device-code relationship |
| FR03 | Search | Exact SKU and model-code queries return the correct result; ambiguous and empty results have useful recovery paths |
| FR04 | Product filtering | Customers can filter by relevant part attributes and availability without losing selected-device context |
| FR05 | Variant selection | Changing a variant updates SKU, imagery where relevant, specifications, price, availability, and compatibility |
| FR06 | Delivery estimate | Postcode checks disclose supported delivery options and any conditions or uncertainty |
| FR07 | Cart | Quantities can be changed or removed; current price and sellable stock are revalidated before payment |
| FR08 | Checkout | The buyer sees line totals, discounts, delivery charges, applicable tax presentation, and final payable total |
| FR09 | Accessible navigation | Finder, filters, product selection, cart, and checkout work by keyboard and have meaningful labels |
| FR10 | Order access | Customers can securely retrieve their own orders and tracking details through the chosen account or verified access flow |
| FR11 | Customer support | Help is available from relevant product and order pages; business contact details are accurate |
| FR12 | Production content | No demonstration pricing, copied identity, fake reviews, placeholder policies, or inactive purchase actions remain |

### Search and compatibility rules

Treat product discovery and compatibility verification as different operations. A search suggestion can be approximate; a confirmed compatibility result must use an approved relationship. Preserve punctuation-insensitive and case-insensitive aliases while keeping distinct model codes separate. A similar product title must never create a compatibility relationship automatically.

### Content management

Staff should be able to maintain category descriptions, FAQs, model-identification help, delivery explanations, and policy pages without a code deployment. Product specifications should come from structured data so the same facts appear consistently in cards, details, filters, and order records.

# 6 Order and administration requirements

| ID | Requirement | Acceptance condition |
|---|---|---|
| FR13 | Catalogue administration | Staff can create, review, publish, suspend, and archive products without deleting historical order information |
| FR14 | Inventory control | Receipts, reservations, sales, releases, returns, and adjustments produce traceable stock changes |
| FR15 | Payment verification | Only a verified payment outcome or an approved COD workflow can move an order into fulfilment |
| FR16 | Duplicate-event handling | Repeated provider events do not create duplicate orders, stock deductions, labels, refunds, or messages |
| FR17 | Shipment processing | Staff can create or retry shipments safely and record carrier references and tracking events |
| FR18 | Returns and refunds | Each request links to an order item; eligibility, inspection, refund, and replacement states are recorded separately |
| FR19 | Customer notifications | Transactional messages follow verified order events; delivery failures are visible to staff |
| FR20 | Staff permissions | Catalogue, warehouse, support, and finance access is limited to the tasks each role needs |
| FR21 | Audit history | Material price, compatibility, stock, refund, and permission changes record actor, time, and reason |
| FR22 | Reconciliation | Staff can identify orders whose payment, shipment, inventory, or refund states disagree |

### Order status design

Keep order status, payment status, and fulfilment status as separate fields. For example, a confirmed order may be paid but unfulfilled; a delivered order may have a partial refund under review. One overloaded status field cannot represent these cases safely.

Proposed payment states are pending, authorised where supported, paid, failed, partially refunded, and refunded. Proposed fulfilment states are unfulfilled, processing, partially shipped, shipped, delivered, and returned. Exact mappings must follow the selected platform and providers rather than introduce conflicting parallel states.

### Inventory rules

Define when stock is reserved, when a reservation expires, and when a cancelled or failed order releases stock. Handle concurrent buyers of the last available item. If a late payment arrives after stock has been released, place the order into a review path rather than silently oversell. Returned goods remain quarantined until inspection confirms that they are sellable.

### Administrative exceptions

Provide work queues for incomplete catalogue records, unverified compatibility, low stock, failed imports, pending payments, failed shipments, delivery exceptions, and unresolved returns. Each exception needs an owner, last update, next action, and enough history for another team member to continue the work.

# 7 Architecture and integrations

### Recommended launch architecture

Use a custom Shopify theme for the public storefront and Shopify as the authoritative commerce platform for products, variants, customers, orders, and inventory. Build the Device First finder using structured device records and verified variant-to-device mappings. Shopify supports structured metaobjects and references that can be used in themes and applications [1].

The existing Sites demonstration remains a review environment. Moving to this recommended architecture requires a production implementation and catalogue migration; the current static files do not become a live Shopify store automatically.

### Component responsibilities

| Component | Responsibility | Authoritative information |
|---|---|---|
| Public storefront | Render pages, finder, search, product details, and cart | Reads approved catalogue and commerce data |
| Commerce platform | Checkout, orders, customers, stock, and staff administration | Product variants, prices, orders and inventory |
| Compatibility module | Resolve exact devices and verified fit relationships | Canonical devices and approved compatibility records |
| Payment provider | Process authorised payments and refunds | Payment transaction outcome |
| Shipping integration | Serviceability, labels, shipment creation, and tracking | Carrier shipment and tracking references |
| Notification service | Send transactional updates with retry visibility | Message delivery state |
| Search index if required | Fast retrieval and filtering | Derived copy; never the stock or price authority |

Razorpay and Shiprocket are candidate integrations with published Shopify integration documentation [3][4]. Their use is a recommendation, not an account connection or commercial commitment. Verify plan support, onboarding requirements, fees, settlement terms, supported services, and data flows before procurement.

### Integration and deployment controls

Use managed checkout and official provider integrations where they meet the requirement. A custom application, if needed, must keep secrets on the server, verify inbound notifications, record processed event IDs, retry safely, and reconcile missed events. Webhook payloads are notifications to validate, not unrestricted instructions to execute.

Maintain separate development, staging, and production configurations. Use test payment credentials outside production. Version theme and custom application code, review releases, retain a rollback version, and back up application data and configuration. Buying a domain provides the address; commerce hosting, payment processing, and shipping remain separate services.

### Scale decision

Validate the actual catalogue before selecting the finder implementation. Standard Shopify filters have collection and value limits, including no filters on collections exceeding 5,000 products and a maximum of 100 displayed values per filter [2]. Large device directories may need a dedicated lookup or search service. Introduce that service when measured catalogue or performance needs justify it.

# 8 Core data relationships

### Model the business before importing products

These are logical entities, not an instruction to create duplicate database tables for data already managed by Shopify. Map them to native platform records, metafields, metaobjects, or a custom application as appropriate. The implementation must identify one authoritative owner for every field.

| Entity | Key relationships | Purpose |
|---|---|---|
| Brand | One brand has many device models | Standardise manufacturer names and aliases |
| Device model | Belongs to one brand; has one or more exact model codes | Group marketed device names while preserving technical distinctions |
| Device code | Belongs to a model; can have region or revision attributes | Identify the device precisely enough to verify fit |
| Category | Groups products by part type | Support navigation and relevant specification fields |
| Product | Contains one or more sellable variants | Hold shared description, category, and marketing content |
| Variant or SKU | Belongs to a product; has price, stock, and specifications | Identify the exact item sold and fulfilled |
| Compatibility | Links one SKU to one exact device code | Record fit status, evidence, limitations, and approval |

| Entity | Key relationships | Purpose |
|---|---|---|
| Supplier offer | Links a supplier and supplier part number to a SKU | Record cost, lead time, source, and supply terms |
| Stock location | Has balances and stock movements per SKU | Track physical inventory and sellable quantities |
| Customer | Has orders, addresses, and consent preferences | Support service and authenticated order access |
| Order and order item | An order has items, payments, shipments, and returns | Preserve what was purchased and how it was handled |

### Compatibility is many to many

A SKU may fit several exact device codes, and a device code may accept several SKUs. Keep each relationship explicit. Different frame colours, screen technologies, or connector revisions can affect fit, so compatibility should normally resolve at the sellable variant level.

Store evidence and review status alongside the relationship. A supplier's claim can enter a review queue; an approved relationship must identify who reviewed it and when. If evidence changes, record a new review and assess whether affected open orders need attention.

### Stable identifiers and history

Use stable internal IDs and unique SKUs. Names and URLs may change without changing identity. Preserve old aliases and redirects where appropriate. Archiving a product must not remove the purchased description, price, policy reference, or compatibility context from historical order items.

# 9 Catalogue data dictionary

Required means required before publishing or enabling purchase, subject to the category. Optional fields must not be filled with invented values. Use a defined unit and vocabulary for every specification that will be filtered or compared.

| Record | Required fields | Rules and examples |
|---|---|---|
| Brand | brand_id, name, slug | Stable ID; unique slug; aliases kept separately |
| Device model | model_id, brand_id, name, slug | Store marketing name; do not assume all regional versions match |
| Device code | device_code_id, model_id, exact_code | Unique with relevant region or revision; aliases require review |
| Product | product_id, title, category_id, description, status | Status is draft, approved, published, suspended, or archived |
| SKU | sku_id, sku_code, product_id, currency, price, condition, active | Unique SKU; store money in minor units; INR is the launch assumption |

| Record | Required fields | Rules and examples |
|---|---|---|
| SKU specification | sku_id, attribute, value, unit where relevant | Examples: panel type, frame inclusion, colour, connector revision |
| Compatibility | sku_id, device_code_id, status, evidence_reference, reviewer, reviewed_at | Status: verified, unverified, incompatible; confirmed-fit UI uses verified only |
| Media | media_id, product_or_sku_id, asset_url, alt_text, sort_order, rights_reference | Use authorised images; connect variant-specific images to the correct SKU |
| Supplier offer | supplier_id, supplier_part_number, sku_id, cost, currency, source_date | Cost is staff-only; lead time and minimum quantity are separate fields |
| Policy reference | sku_or_category_id, warranty_policy_id, returns_policy_id | Effective dates and version references prevent ambiguous promises |

## Shipping and specification data

Each shippable SKU needs weight and package dimensions, using consistent units. Capture restrictions or handling attributes where relevant. Battery capacity, voltage, display technology, included frame, and tools supplied belong in category-specific structured fields. Do not confuse supplier category labels with validated specifications.

## Example record

The following is illustrative and must not be imported as verified commercial data: product_id PROD-DEMO-001; sku_code SD-DEMO-SCREEN-001; category display-assembly; device_code_id DEVICE-DEMO-A; panel_type OLED; frame_included true; currency INR; price_minor 249900; compatibility_status unverified; publication_status draft.

This example demonstrates field shape only. It asserts no real price, stock, manufacturer origin, or device compatibility. The price_minor value represents INR 2,499.00 in an integer currency field.

## Data constraints

Reject duplicate SKU codes, negative prices, unsupported currencies, missing required specification units, orphan references, and compatibility records without an identified source. Flag stale supplier information for review. A field being present does not make the claim verified; review status remains separate.

# 10 Order and inventory data

## Transaction records

| Record | Main fields | Required invariant |
|---|---|---|
| Order | order_id, customer_reference, currency, totals, order_status, created_at | Final totals come from the commerce platform, not browser input |
| Order item | order_item_id, order_id, sku_id, quantity, price_snapshot, product_snapshot, device_context | Preserve the purchased specification and selected device at order time |
| Payment | payment_id, order_id, provider_reference, amount, currency, status | Provider reference is unique; amount and currency must match the expected transaction |
| Shipment | shipment_id, order_id, carrier_reference, tracking_number, state | Link shipped quantities to order items; allow partial shipments |
| Return request | return_id, order_item_id, quantity, reason, evidence, eligibility_state | Returned quantity cannot exceed the eligible purchased quantity |
| Refund | refund_id, payment_id, return_reference, amount, status | Multiple refunds cannot exceed the refundable balance |
| Reservation | reservation_id, sku_id, location_id, quantity, expires_at, order_reference | Expiry and release are processed once with a traceable reason |
| Stock movement | movement_id, sku_id, location_id, type, quantity, reference, actor, occurred_at | Every adjustment has a source and an audit trail |
| Provider event | provider, event_id, received_at, processed_at, result | A repeated event does not repeat its business side effects |

## Inventory accounting

Track on-hand, reserved, and quarantined quantities separately. A proposed available-to-sell calculation is on-hand minus reserved minus quarantined minus any approved safety buffer. Define whether in-transit inventory is excluded; do not show it as immediately available without an explicit preorder policy.

Use the platform's native inventory controls wherever possible. Any external stock service must synchronise through documented rules and avoid competing with the platform as a second stock authority. Record the reason for manual corrections and reconcile discrepancies against physical counts.

## Customer and access data

Keep customer identifiers, required delivery details, contact information needed for fulfilment, and explicit marketing preferences. Store transaction references rather than raw payment card data. Keep address snapshots on orders so later profile edits do not change the recorded delivery instruction.

### Timestamps and deletion

Store event timestamps consistently, preferably in UTC, and present operational dates in the business timezone. Define retention separately for orders, financial records, support attachments, security logs, and marketing data. Deleting or anonymising a customer profile must follow the approved retention policy without silently corrupting accounting or order records.

# 11 Data sourcing and publication

### Sources the business must provide

Obtain supplier catalogues, SKU and part-number lists, specification sheets, compatibility evidence, authorised product images, purchase costs, stock quantities, shipping dimensions, and warranty terms. Obtain business identity and support details from the founder, not from the reference website. Use carrier and payment providers for their own transaction and service data.

Reference-store imagery and demonstration products must be replaced or authorised before commercial use. Do not import another merchant's reviews, stock claims, warranty promises, or customer data as SpareDukaan records.

### Publication workflow

1. Receive a source file and record supplier, date, version, and permission or ownership information.
2. Import into a staging area with an import batch ID. Preserve the original source separately from transformed records.
3. Normalise brand aliases, model codes, categories, units, currencies, and part terminology.
4. Match existing IDs and detect duplicates without overwriting unrelated products.
5. Validate required fields, prices, references, image links, and shipping data. Produce a row-level error report.
6. Review compatibility evidence, specifications, condition, and policy applicability. Unverified items remain unpublished or clearly restricted.
7. Publish the approved subset and update any derived search index. Record who approved the batch.
8. Reconcile published record counts and sample products against the source; retain a reversible change log.

### Starter import files

| File | Minimum columns |
|---|---|
| brands.csv | brand_id, name, slug, aliases |
| devices.csv | model_id, brand_id, model_name, device_code_id, exact_code, region, revision |
| products.csv | product_id, title, category_id, description, status |
| variants.csv | sku_id, sku_code, product_id, price_minor, currency, condition, weight_g, active |
| compatibility.csv | sku_id, device_code_id, status, evidence_reference, reviewer, reviewed_at |
| inventory.csv | sku_id, location_id, on_hand, quarantined, counted_at, source_reference |
| media.csv | media_id, product_id, sku_id, asset_url, alt_text, sort_order, rights_reference |

These are proposed interchange formats. The developer must map them to the final platform's supported import schema. Do not assume that these custom columns can be uploaded directly through a native product-import screen.

### Data ownership

The catalogue manager owns product completeness and compatibility review. Warehouse staff own physical counts and receipts. Finance owns cost and payment reconciliation. The founder approves commercial policy and customer-facing claims. Assign named people before production import.

# 12 Operations and responsibilities

### Procurement and receiving

Record supplier purchase references and expected deliveries. On receipt, count items, inspect condition, verify identifying labels, and compare the received part with the approved SKU specification. Place suspect or damaged items in quarantine. Publish available stock only after the receiving process accepts the goods.

### Picking and shipping

Release an order for fulfilment only when its payment or approved COD conditions are satisfied. Pick the exact SKU and variant shown on the order. Check quantity, condition, packaging, and included components. Record the shipment reference and trigger customer updates from verified shipment events.

### Returns and replacements

Support opens a request against an order item. Staff record the reason, relevant evidence, policy version, eligibility decision, and customer communication. Received items are inspected before any restock. Replacement orders and refunds retain links to the original order and request so the financial and stock effects can be reconciled.

### Role permissions

| Role | Allowed work | Restricted work |
|---|---|---|
| Founder or administrator | Approve policies, configure services, assign roles, review reports | High-risk changes should still be logged |
| Catalogue manager | Edit products, evidence, images, and compatibility | Payment secrets and unrestricted refunds |
| Warehouse operator | Receive, count, pick, pack, and record shipment handling | Customer marketing exports and pricing policy |
| Support agent | View necessary order details and manage support requests | Broad customer exports and unrestricted payment changes |
| Finance operator | Reconcile payments, settlements, and approved refunds | Unnecessary catalogue or access administration |
| Developer | Maintain approved application configuration and releases | Routine access to production customer data without a defined need |

### Daily and weekly work

Daily, review unresolved payments, shipment failures, delayed orders, return requests, and low-stock exceptions. Weekly, review catalogue errors, compatibility complaints, physical stock discrepancies, supplier issues, and fulfilment performance. Assign an owner and next action to each unresolved issue.

### Customer communication

Use clear confirmation, shipment, delivery-exception, return-decision, and refund messages. Keep marketing opt-in separate from order notifications. If WhatsApp or SMS is introduced, configure the provider and consent process deliberately. A support button should lead to a real supported channel with defined service hours.

# 13 Security and quality requirements

### Access and privacy

Require appropriate staff authentication and use multifactor authentication where supported. Apply least-privilege roles. Customers must only access their own orders and support records. Verify server-side authorisation for every protected operation; hiding a button is not access control.

Keep secrets in the platform or server secret store, never in public scripts, exported catalogues, or source control. Validate and sanitise submitted content. Restrict upload types and sizes, scan attachments where supported, and avoid exposing private support images through public catalogue storage.

### Reliability and recovery

Record integration failures without exposing secrets or unnecessary personal details. Retry temporary failures with limits and an operator-visible final failure state. Maintain recoverable backups of custom data, theme code, and configuration. Test restoration and rollback before launch; confirm which native records are covered by the chosen platform and backup service.

### Performance and accessibility

The following are proposed acceptance targets, not claims about the current prototype. On representative mobile devices and networks, aim for a main-content load within 2.5 seconds, interaction response within 200 milliseconds, and minimal layout movement. Confirm the measurement method and supported device matrix before making these contractual targets.

Optimise product images, limit third-party scripts, cache safe catalogue responses, and avoid downloading the entire catalogue into the browser. Core browsing and checkout should work with keyboard navigation, readable contrast, meaningful form labels, and screen-reader announcements for important state changes. Aim for WCAG 2.2 AA and include manual testing.

### Search visibility

Give brand, model, category, and product pages stable URLs and useful original descriptions. Use canonical URLs, appropriate redirects, a sitemap, and truthful product metadata. Avoid indexing staging pages, private customer content, or large numbers of duplicate filter combinations. Never publish fake ratings or unavailable offers in structured data.

### Operational safeguards

Revalidate price and stock before accepting the order. Keep production payment and shipping settings separate from test settings. Review custom-domain DNS and certificate setup before redirecting public traffic. Configure tax display, invoices, shipping terms, privacy, and returns with the business's responsible advisers before commercial launch.

# 14 Testing and launch acceptance

## Minimum test scenarios

| Test | Expected result |
|---|---|
| Two similar model names have different exact codes | The finder distinguishes them and does not infer compatibility |
| A SKU has an unverified compatibility record | No confirmed-fit claim appears |
| A buyer changes a product variant | SKU, specifications, price, stock, and fit information update together |
| Two buyers attempt to buy the last item | The configured stock policy prevents unsupported overselling |
| A payment notification is repeated | The order, stock, and message effects occur once |
| A successful payment arrives late | The order is reconciled against stock and expiry rules |
| Shipment creation fails after payment | The paid order remains visible in an actionable fulfilment queue |
| A customer opens another customer's order link | Access is denied without leaking personal information |
| A partially refunded order is refunded again | The refundable balance is enforced |
| A returned item is received | It remains quarantined until inspection approves restocking |
| An import contains duplicates or broken references | Invalid rows are rejected with useful errors |
| A mobile buyer uses only keyboard or assistive navigation | Core discovery and checkout controls remain usable |

## Release acceptance

The founder approves business identity, contact details, policies, opening assortment, pricing, and delivery rules. Catalogue staff approve compatibility evidence and representative product records. Operations demonstrate receiving, picking, shipment tracking, and returns. Finance demonstrates payment and refund reconciliation. The developer demonstrates access controls, error recovery, backups, and rollback.

Use test-mode transactions first. Any live payment or refund test must be deliberately authorised, limited, and reconciled. Test both the customer-visible result and the underlying provider and platform records; a success screen alone is insufficient evidence.

### Launch blockers

Do not launch with unverified fit claims, placeholder policy pages, test credentials, broken checkout, uncontrolled overselling, exposed secrets, inaccessible order records, or an unresolved payment-to-order reconciliation defect. Critical and high-severity issues affecting purchase correctness, access control, or financial handling must be resolved before release.

### Launch sequence

Freeze and validate the approved catalogue; back up the release; deploy production configuration; verify domain and certificate; perform authorised end-to-end checks; open to a limited audience; monitor exceptions; then broaden acquisition. Keep an agreed rollback and customer communication plan available during the launch window.

# 15 Delivery plan and business decisions

### Phased delivery

| Phase | Deliverables | Exit condition |
|---|---|---|
| Foundation | Confirm customers, stock model, opening catalogue, budget, platform, and policies | Founder approves scope and named data owners |
| Catalogue proof | Import a representative sample with model codes, variants, evidence, images, and stock | Finder and data mapping work on difficult cases |
| Storefront build | SpareDukaan design, navigation, finder, search, product pages, and cart | Core customer journeys pass review on mobile and desktop |
| Commerce integration | Payments, shipping, accounts, order access, returns, and staff workflows | End-to-end order and exception tests pass |
| Launch preparation | Content completion, access checks, monitoring, recovery, and domain setup | Release acceptance is recorded |
| Expansion | Trade features, deeper catalogue, improved search, and automation | Demand and operational evidence justify each addition |

Calendar dates and prices should be estimated after the catalogue proof. The difficulty of device compatibility, supplier data quality, and operational integrations can change the scope substantially. No fixed delivery timeline or implementation price is committed by this document.

### Cost categories to budget

Budget for the domain, commerce subscription, theme or custom development, finder/search functionality, paid applications, payment transaction fees, shipping and return costs, communication charges, catalogue preparation, product photography, stock, support staffing, and ongoing maintenance. Separate one-time setup costs from recurring fixed costs and per-order costs.

### Decisions required from the founder

1. Is the launch aimed primarily at retail buyers, repair shops, or both?

2. Will products be stocked, supplied on demand, or fulfilled through a mixed model?

3. Which suppliers, brands, phone models, categories, and SKUs are in the opening assortment?
4. What evidence is available for compatibility, condition, warranty, and image usage?
5. Who owns catalogue review, warehouse work, support, and finance reconciliation?
6. What are the budget, launch window, delivery regions, shipping rules, and COD policy?
7. Has sparedukaan.com been purchased, and who controls the registrar and DNS account?
8. Which commerce platform, payment provider, shipping provider, and communication channels are approved?

### Main delivery risks

Poor compatibility data can cause wrong-part purchases; test difficult model variants early. Inaccurate supplier stock can cause overselling; define stock authority and update frequency. Unclear warranty language can create disputes; approve product-specific terms before publishing. Extensive custom checkout or multi-seller features can expand cost and maintenance; keep them outside the agreed initial scope.

# 16 Measurement and operating reviews

### What to measure

Use measurement to identify customer confusion and operational faults. The first month should establish a baseline; commercial targets should be approved after the team has reliable data. The definitions below are proposed operating measures, not existing business results.

| Measure | Proposed definition | Decision it supports |
|---|---|---|
| Finder completion | Sessions that select an exact device divided by sessions that start the finder | Whether device discovery is understandable |
| Search no-result rate | Valid searches returning no products divided by valid searches | Missing catalogue coverage or search vocabulary |
| Purchase conversion | Sessions with a verified purchase divided by eligible storefront sessions | Whether discovery and checkout lead to orders |
| Payment completion | Successfully paid payment attempts divided by initiated attempts | Provider or checkout friction; retries need consistent counting |
| Compatibility-related returns | Delivered order items with an approved wrong-fit return divided by delivered items in a defined cohort | Quality of compatibility evidence and customer selection |
| Dispatch reliability | Eligible orders dispatched within the promised handling period divided by eligible orders | Whether operating promises match capacity |
| Contribution per order | Net revenue less defined variable product and fulfilment costs | Whether sales create sustainable contribution |

### Event design

Proposed events include device_finder_started, device_selected, search_submitted, search_no_results, product_viewed, variant_selected, add_to_cart, checkout_started, purchase_verified, shipment_dispatched, and return_requested. Record stable product, SKU, device, and order references where appropriate. Do not send passwords, full addresses, payment credentials, or unnecessary personal information to analytics.

Use verified order records to reconcile purchase events and prevent double counting. Establish a single definition of eligible sessions, test traffic, bot exclusions, reporting timezone, and attribution window before comparing performance. Search text can contain personal information, so apply a minimisation and retention rule.

### Review cadence

Review failed payments and fulfilment exceptions daily. Review search failures, catalogue gaps, stock discrepancies, and return reasons weekly. Review margins, supplier performance, and product expansion decisions monthly. Record actions and owners rather than collecting metrics without a response.

### Expansion criteria

Add wholesale functions when there is sustained repair-shop demand and clear pricing rules. Add a dedicated search service when actual catalogue size or measured search performance requires it. Expand device coverage when the team can verify compatibility and support the resulting orders. Avoid using raw catalogue size as a substitute for customer success.

# 17 References and glossary

### Product context

The founder selected SpareDukaan as the name and chose the Device First concept in the design comparison. The production requirements in this document are recommendations for implementation. Budget, launch dates, catalogue ownership, suppliers, and provider accounts require confirmation.

The earlier private demonstration is available at:

https://mobile-parts-store-24oct.er-faiz007.chatgpt.site

This link points to the reference-style prototype, not a completed SpareDukaan production store. Domain registration availability changes; this document does not establish ownership of sparedukaan.com.

### Official technical references

[1] Shopify Help Center. Metaobjects. Structured custom data and references for store content. Accessed 25 October 2026.

https://help.shopify.com/en/manual/custom-data/metaobjects

[2] Shopify Help Center. Adding filters with Shopify Search and Discovery. Requirements and collection/filter limits. Accessed 25 October 2026.

https://help.shopify.com/en/manual/online-store/storefront-search/search-and-discovery-filters

[3] Razorpay Documentation. Shopify Plugin Prerequisites. Integration information for the All-in-one Razorpay Payment Gateway. Accessed 25 October 2026.

https://razorpay.com/docs/payments/payment-gateway/ecommerce-plugins/shopify/

[4] Shiprocket. Shopify integration. Order and shipping integration overview. Accessed 25 October 2026.

https://www.shiprocket.in/channel-integrations/shopify/

Provider capabilities, limits, plan requirements, and charges can change. Verify the implementation-relevant details again before contracting or deploying. The references support platform capabilities; they do not prove that any account or integration has been configured for SpareDukaan.

## Glossary

| Term | Meaning |
|---|---|
| SKU | Unique identifier for the exact sellable variant |
| Variant | A purchasable version of a product with its own attributes and commercial data |
| Compatibility record | Evidence-backed link between a SKU and an exact device code |
| Metaobject | A structured custom content record in Shopify |
| Webhook | A notification from one service to another about an event |
| Idempotency | Processing repeated requests or events without repeating their business effect |
| Reservation | Stock temporarily held for a purchase or order |
| Quarantine | Stock withheld from sale pending inspection or resolution |
| COD | Cash on delivery, subject to approved business and carrier rules |
| MVP | The smallest production release that completes the agreed customer and operating journeys |